Retrieval-Augmented Generation: Questions and Answers

Notes on Lewis et al. (2020)

1 Parametric and Non-Parametric Memory

Question: What is "parametric and non-parametric memory" in this context?

	Parametric memory	**Non-parametric memory**
What it is in RAG	The weights θ of the BART generator (400M parameters)	The document index: 21M Wikipedia passages plus their vectors
Where knowledge lives	Implicitly, spread across the weights, learned during pre-training	Explicitly, as readable text outside the model
How to update it	Retrain or fine-tune	Replace the index; no training needed ("index hot-swapping", §4.5)
Can you inspect it?	No	Yes: you can see which passage was used, which gives provenance

A good analogy: parametric memory is what a student has **memorized** (closed-book exam). Non-parametric memory is the **textbook they bring** (open-book exam).

"Non-parametric" is used in the same sense as k-nearest-neighbours. The model does not compress the data into a fixed set of parameters. It keeps the data and looks things up, so the memory can grow or change without changing the model's size.

2 RAG vs. LoRA

Question: Do RAG and LoRA have the same goal, namely letting the model access knowledge outside the data it was trained on?

Not quite. They overlap a little, but they solve different problems:

- **RAG** keeps knowledge **outside** the model and fetches it at inference time. The model gets access to information it never saw in training.
- **LoRA** is a **cheap way to fine-tune**. It freezes the original weights W and learns a small low-rank update $\Delta W = BA$. Whatever it is trained on ends up **inside the parameters**, so LoRA is still parametric memory. The new data simply becomes more training data.

	RAG	**LoRA**
Where knowledge goes	External index	Into the (adapter) weights
Adding new facts	Add documents to the index	Collect data and train again
Can cite sources	Yes	No
Typically good at	Facts, fast-changing or private knowledge	Adapting behavior, style, format, a task or domain

In short, LoRA is like studying for the exam, and RAG is like bringing the book. Later studies (e.g. Ovadia et al. 2023, "Fine-Tuning or Retrieval?") found that fine-tuning is fairly unreliable for injecting new facts, while retrieval works better for that.

The two are **complementary**. In this paper the generator and query encoder are fine-tuned (fully, since LoRA came out a year later) so the model learns **how to use** retrieved documents, not to memorize the facts. Today LoRA is often used for exactly that fine-tuning step of a RAG system.

3 What a "Document" Is

Question: What is the "document" in this context?

A **document z is a 100-word passage of Wikipedia, not a whole article.** From §3: the authors take the December 2018 Wikipedia dump and split each article into **disjoint 100-word chunks**, which gives about **21 million documents**. Each chunk is encoded by BERT_d into a vector and stored in the FAISS index.

The paper uses "document" and "passage" interchangeably. In today's RAG terminology this is a **chunk**. Small chunks are used for two reasons:

- The encoder produces a more precise vector for a short, focused passage.
- The passage has to fit in BART's input together with x, because they are simply concatenated (§2.3).

Real examples appear in Figure 2 (page 7): "Document 1: ... His wartime experiences formed the basis for his novel 'A Farewell to Arms' ..." is one such 100-word chunk.

4 Retriever and Retrieve

Question: What is "retriever" and "retrieve" in this context?

Retrieve means: given an input x (a question, a claim, ...), look up the most relevant pieces of text in a large external collection (here, Wikipedia). It works like a search engine.

The **retriever** is the component that does this search. The paper writes it as $p_\eta(z \mid x)$: a probability distribution over documents z given the input x. It is built on DPR (Dense Passage Retriever, Section 6), which uses two BERT encoders (Section 5; paper §2.2):

- **Query encoder** $\mathbf{q}(x) = \text{BERT}_q(x)$ turns the input into a vector.
- **Document encoder** $\mathbf{d}(z) = \text{BERT}_d(z)$ turns each Wikipedia passage into a vector. This is done once, ahead of time, and stored in an index.

Retrieval then means finding the K documents whose vectors have the highest dot product with the query vector:

$$\text{top-}k\big(p_\eta(\cdot \mid x)\big) = \text{the } K \text{ documents } z \text{ with the largest } \mathbf{d}(z)^\top \mathbf{q}(x). \tag{1}$$

This search is called **MIPS** (Maximum Inner Product Search). The paper uses FAISS so it stays fast over 21M documents.

5 BERT

Question: What is BERT in this context?

BERT stands for *Bidirectional Encoder Representations from Transformers* (Devlin et al., 2018). It is a neural network that **reads a text and turns it into a vector**, a list of numbers that captures what the text means. It only *understands* text. It does **not** write text.

Three things to know:

- **Encoder.** Text goes in, vectors come out. (BART in Section 7 also has a decoder, which writes text. BERT has none.)
- **Bidirectional.** It reads the whole text at once and looks at the words on the left *and* on the right of every word. So "bank" gets a different vector in "river bank" than in "bank account".
- **Pre-trained by fill-in-the-blank.** About 15% of the words are hidden, and BERT learns to guess them from the other words. This needs a lot of text but no labels.

The paper uses the small version, $\text{BERT}_{\text{BASE}}$: 12 layers, a vector of 768 numbers per text, and 110M parameters (Appendix G).

Where BERT appears in RAG. BERT is not the whole retriever. It is a **building block inside DPR** (Section 6). RAG uses two separate copies of it (§2.2):

- $\mathbf{q}(x) = \text{BERT}_q(x)$ turns the input (for example a question) into a vector. It is **fine-tuned** during RAG training.
- $\mathbf{d}(z) = \text{BERT}_d(z)$ turns each Wikipedia passage into a vector. It stays **frozen** (§2.4).

Each text becomes one vector (DPR takes the output at the special `[CLS]` token at the start of the text, which acts as a summary of the whole text). Texts with a similar meaning get similar vectors, so a large dot product means "these two texts are about the same thing". For example, the question "Who wrote A Farewell to Arms?" and the passage about Hemingway (Figure 2 of the paper) get similar vectors, even if they share few words.

What BERT does not do. It does not write the answer. In RAG, writing is the job of BART.

A simple picture: BERT is a **reader who writes a short "meaning fingerprint"** for any text. Two texts with similar fingerprints are about similar things.

6 DPR

Question: What is DPR in this context?

DPR stands for **Dense Passage Retriever** (Karpukhin et al., 2020). It is a method for **finding the passages that help answer a question**. In RAG, DPR *is* the retriever $p_\eta(z \mid x)$ (§1 and §2.2).

The name explains the idea:

- **Passage**: a 100-word chunk of Wikipedia (Section 3).
- **Dense**: every text becomes a dense vector (768 numbers) made by a neural network. The older way (TF-IDF, BM25) is *sparse*: it only counts matching words. A dense vector matches **meaning**, so "Who wrote. . . ?" can match a passage that says "authored by. . . ".
- **Retriever**: it looks things up, it does not write.

How it works. DPR is a **bi-encoder**, which means two BERT encoders (Section 5):

1. **Offline, once:** BERT_d turns all 21M passages into vectors, which are stored in a FAISS index.
2. **Online, for each input:** BERT_q turns the input x into a vector $\mathbf{q}(x)$.

3. **Search (MIPS):** return the K passages with the largest dot product $\mathbf{d}(z)^\top \mathbf{q}(x)$.

The score becomes a probability with $p_\eta(z \mid x) \propto \exp\big(\mathbf{d}(z)^\top \mathbf{q}(x)\big)$.

Why two separate encoders? A passage does not depend on the question, so all passages can be encoded **ahead of time**. At question time only the question is encoded, and this is why searching 21M passages is fast. The price is that the question and the passage never "see" each other inside the network, so the match is less exact than with a *cross-encoder*, which reads both together but is far too slow to run on 21M passages. The original DPR question-answering system adds such a cross-encoder to re-rank the results. The RAG paper shows that this extra step is not needed (§4.1).

How DPR was trained. Training data are questions, each with a passage that contains the answer (a *positive*) and passages that do not (*negatives*). The two encoders learn to give the question a higher dot product with the positive than with the negatives. RAG starts from a DPR trained this way on Natural Questions and TriviaQA (§2.2). During RAG training only the query encoder is updated. The document encoder and the index stay frozen (§2.4). The index built with the document encoder is the **non-parametric memory** (Section 1).

A simple picture: DPR is a **librarian who understands meaning**. Every passage sits at a spot on a big map, and passages with similar meaning sit close together. The question is placed on the same map, and the librarian grabs the K closest passages. The librarian only **fetches**. Someone else has to read them and write the answer.

7 BART

Question: What is BART in this context?

BART stands for *Bidirectional and Auto-Regressive Transformers* (Lewis et al., 2019). It is a **sequence-to-sequence (seq2seq)** model: it reads one text and **writes another text**. In RAG it is the **generator** $p_\theta(y_i \mid x, z, y_{1:i-1})$ (§2.3). The paper uses BART-large, with about 400M parameters (406M in Appendix G). Its weights θ are the **parametric memory** (Section 1).

BART has two parts:

- **Encoder** (like BERT). It reads the whole input, looking left and right of every word ("Bidirectional").
- **Decoder** (like GPT). It writes the output **one token at a time, from left to right**. Each new token depends on the tokens already written ("Auto-Regressive") and on what the encoder read.

So BART is roughly a **BERT-style reader plus a GPT-style writer**.

How it was pre-trained. BART learns by **denoising**. Clean text is damaged (words are hidden or deleted, sentences are shuffled), and BART must rebuild the original. This is like fixing a scrambled sentence. It makes BART good at writing fluent text, and it reached state-of-the-art results on many generation tasks (§2.3).

BART inside RAG.

- **Input:** the original input x and a retrieved passage z are simply **concatenated** and given to the encoder (§2.3).
- **Output:** the decoder writes the answer y token by token.
- BART runs once for each of the K retrieved passages, and the results are combined by marginalization (see Section 10).
- **All of BART is fine-tuned** during RAG training.

Example: the input is "Who wrote A Farewell to Arms?" plus the Hemingway passage, and BART writes "Ernest Hemingway". It can also write a sentence or a paragraph in its own words. It is not limited to copying text from the passage.

A simple picture: BART is the **student who reads the question and the fetched pages, then writes the answer**.

8 DPR vs. BART

Question: What is the difference between DPR and BART?

Short answer: DPR **finds** text and BART **writes** text. They are two different parts of RAG, and they work one after the other:

$$x \xrightarrow{\text{DPR: search}} z_1, \ldots, z_K \xrightarrow{\text{BART: read } x+z_i, \text{ write}} y.$$

	DPR	**BART**
Job in RAG	Retriever: finds the top-K passages	Generator: writes the answer
Symbol in the paper	$p_\eta(z \mid x)$	$p_\theta(y_i \mid x, z, y_{1:i-1})$
Built from	Two $\text{BERT}_{\text{BASE}}$ encoders (a bi-encoder), no decoder	One encoder–decoder Transformer (BART-large)
Size in the paper	110M per encoder	about 406M
Input	The question x or a passage z, encoded *separately*	x and z *together* (concatenated)
Output	One vector per text; the score is a dot product	Text, one token at a time
Can it write new text?	No. It can only return existing passages	Yes
Pre-training	BERT: guess hidden words. Then DPR: question–passage pairs	Denoising: rebuild damaged text
Memory in RAG	Gives access to the index (non-parametric memory)	Its weights θ are the parametric memory
Trained in RAG?	Only BERT_q. BERT_d and the index stay frozen	All of BART

What they have in common. Both are Transformers, both were pre-trained on a lot of text before RAG, and both contain a BERT-style encoder that reads in both directions. The difference is what is built on top. DPR is **two encoders**: it can turn text into vectors and compare them, but it has **no decoder**, so it cannot write. BART is **an encoder plus a decoder**: it can write, but it has **no search**. It only knows what is stored in its weights and what is put into its input.

Why RAG needs both. BART alone is a closed-book exam: it can only use what it memorized. DPR is what lets it take an open-book exam. This also shows how each part can fail:

- If DPR fetches the **wrong passages**, BART may write a fluent but wrong answer.

- If BART is weak, the passages may be **right** but the answer is poorly written or misses the point.

9 Figure 1, and the Meaning of p and q

Question: Explain Figure 1. What are p and q, and what is the difference between them?

Reading the figure from left to right:

1. **Input** x is one of three task examples: a question ("Define middle ear"), a claim to verify ("Barack Obama was born in Hawaii."), or a Jeopardy answer ("The Divine Comedy").
2. **Query encoder** $\mathbf{q}(x)$ turns x into a vector.
3. **MIPS over the document index** compares that vector with every precomputed $\mathbf{d}(z)$ and returns the top-K documents $z_1, \ldots, z_4$. Steps 2–3 together form the retriever p_η, labelled "Non-Parametric".
4. **Generator** p_θ (BART, labelled "Parametric") takes x concatenated with each z_i and generates the output y.
5. **Marginalize** combines the predictions from all K documents into one final answer, weighted by how relevant each document is.
6. **"End-to-End Backprop through** q **and** p_θ**"**: during training, gradients update the query encoder and BART. The document encoder $\mathbf{d}(z)$ and the index stay **frozen** (§2.4), because re-encoding 21M documents after every update would be too expensive.

The difference between p and q.

- p **is always a probability distribution.** The subscript tells you which parameters it uses:
  - $p_\eta(z \mid x)$: the retriever. It gives the probability that document z is relevant to x; η are the retriever's parameters.
  - $p_\theta(y_i \mid x, z, y_{1:i-1})$: the generator. It gives the probability of the next token; θ are BART's parameters.
- **q is not a probability.** It is the **query encoder function**, which outputs a vector. It is one ingredient of p_η:

$$p_\eta(z \mid x) \;\propto\; \exp\big(\mathbf{d}(z)^\top \mathbf{q}(x)\big). \tag{2}$$

So $\mathbf{q}$ (and $\mathbf{d}$) produce the vectors, and p_η turns their dot-product scores into probabilities (a softmax over the retrieved documents).

If you have seen q in VAEs, where it means an approximate posterior: that is *not* its meaning here.

10 The RAG-Sequence and RAG-Token Equations

Question: Explain the equations for RAG-Sequence and RAG-Token.

In both models the retrieved document z is treated as a **latent variable**. We do not know which document is "correct", so we sum over the top-K candidates, weighting each by the retriever's probability. This is the law of total probability:

$$p(y \mid x) = \sum_z p(z \mid x)\, p(y \mid x, z). \tag{3}$$

RAG-Sequence.

$$\begin{aligned} p_{\text{RAG-Sequence}}(y \mid x) &\approx \sum_{z \in \text{top-}k} p_\eta(z \mid x)\, p_\theta(y \mid x, z) \\ &= \sum_{z \in \text{top-}k} p_\eta(z \mid x) \prod_{i=1}^{N} p_\theta(y_i \mid x, z, y_{1:i-1}). \end{aligned} \tag{4}$$

Here the **sum is outside the product**. For each document, generate the whole answer using only that document (the product over tokens i). Then take a weighted average of those full-sequence probabilities. The assumption is that one document is responsible for the entire output.

RAG-Token.

$$p_{\text{RAG-Token}}(y \mid x) \approx \prod_{i=1}^{N} \sum_{z \in \text{top-}k} p_\eta(z \mid x)\, p_\theta(y_i \mid x, z, y_{1:i-1}). \tag{5}$$

Here the **sum is inside the product**. At each token, mix the next-token distributions from all K documents, pick the token, then move on. Different tokens can be supported by different documents.

A small numeric example. Take two documents with $p_\eta(z_1 \mid x) = 0.7$ and $p_\eta(z_2 \mid x) = 0.3$, and a 2-token answer $y = (y_1, y_2)$. Token y_1 is well supported by z_1 (0.9 vs. 0.2). Token y_2 is well supported by z_2 (0.9 vs. 0.1).

$$\text{RAG-Sequence:} \quad 0.7 \cdot (0.9 \cdot 0.1) + 0.3 \cdot (0.2 \cdot 0.9) = 0.063 + 0.054 = 0.117, \tag{6}$$

$$\text{RAG-Token:} \quad (0.7 \cdot 0.9 + 0.3 \cdot 0.2) \cdot (0.7 \cdot 0.1 + 0.3 \cdot 0.9) = 0.69 \cdot 0.34 \approx 0.235. \tag{7}$$

RAG-Token gives this answer about twice the probability, because it can **combine information from several documents**. That is exactly the paper's Figure 2 (the Hemingway example): "The Sun Also Rises" comes from Document 2 and "A Farewell to Arms" comes from Document 1. It is also why RAG-Token does best on Jeopardy question generation (§4.3).

Details worth knowing.

- In **both** models, retrieval happens **once per input**. RAG-Token does not search again for each token. It reuses the same K documents and re-weights them per token.
- For classification (e.g. FEVER), the output is a single token ($N = 1$), so the two formulas become identical.
- Training minimizes $-\sum_j \log p(y_j \mid x_j)$. Because $p_\eta(z \mid x)$ sits inside that formula, the retriever learns which documents help, even with **no labels saying which document is correct**.
- For decoding (§2.5), RAG-Token works with a normal beam search. RAG-Sequence needs a separate beam search per document ("Thorough" or "Fast" decoding).